Guía de secuencia — Laboratorio TRITON

Del clonado del repositorio al taller listo para impartirse

Taller: Análisis forense OT con asistencia de IA **Evento:** CyberForce Summit 2026 · Track TECH-COM **Duración del taller:** 2 h

Modelo: local, sin cuentas ni internet **Cadena verificada:** septiembre 2026, Ubuntu 24.04

> **La regla que decide si esto funciona**
>
> Todo el trabajo se hace con un **usuario sin privilegios**. Nunca root, nunca `sudo`. Wireshark deshabilita los scripts Lua cuando corre como superusuario, el disector de TriStation no carga, y **no hay mensaje de error**: simplemente verás `UDP` en lugar de `TRISTATION` y el taller se queda sin sustancia. Es la falla número uno y cuesta media hora encontrarla en vivo.

0 Antes de empezar

10 min

Reúne esto antes de tocar la terminal. Nada de lo que sigue funciona sin ello.

- [ ] Hipervisor instalado y con espacio (VirtualBox, VMware, UTM o Proxmox)
- [ ] ISO de **Ubuntu Server 24.04 LTS** descargada — es la versión con la que se verificó todo
- [ ] Acceso al repositorio privado `cyberforcesummit2026_workshop`
- [ ] Credenciales de GitHub funcionando (token o SSH) — pruébalas antes
- [ ] Una máquina anfitriona con **holgura para asignarle 8 GB de RAM** a la VM
- [ ] Red decente para la descarga inicial: **son ~5 GB**, la mayoría pesos del modelo
- [ ] Al menos **2 horas libres** para llegar hasta el ensayo sin interrupciones

> **Lo que ya no necesitas**
>
> Ni cuentas de proveedores de IA, ni tokens, ni que el venue deje pasar ningún dominio. Con el modelo en local, la única red que hace falta es la de la preparación.

1 Construir la VM

25 min

Recurso	Mínimo	Recomendado	Por qué
vCPU	4	8	La inferencia en CPU escala con núcleos
RAM	8 GB	16 GB	El modelo ocupa ~4 GB residentes
Disco	30 GB	40 GB	Pesos, Ollama, sistema y evidencia
Red	NAT con internet		Sólo durante la preparación

Crear el usuario del laboratorio

No es cosmético. Es requisito funcional, por lo que dice el recuadro rojo de arriba.

```
sudo adduser analista
sudo usermod -aG sudo analista
su - analista
```

A partir de aquí, **todo se hace como `analista`**.

2 Clonar y provisionar

15 min

```
cd ~
git clone https://github.com/gustavo2107/cyberforcesummit2026_workshop.git taller
cd taller
make bootstrap
```

`make bootstrap` hace todo de corrido y es idempotente — si algo falla a media, lo vuelves a correr sin romper nada. **Descarga unos 5 GB**, así que hazlo con red decente y nunca en el venue:

- Se niega a correr si eres root, y te explica por qué
- Instala `tshark` respondiendo **No** a la pregunta del setuid (aquí sólo se leen archivos PCAP, nunca se captura de la interfaz)
- Descarga la evidencia desde el repositorio de Nozomi e imprime las huellas SHA-256
- Instala el disector en la ruta de plugins que le pregunta a tu propio binario, sin asumirla
- Instala Ollama, descarga el modelo y deja listos el cliente (`ollmcp`) y el servidor (`wireshark-mcp`)
- Corre la verificación al final

Si vas a usar un proveedor en la nube en lugar del modelo local: `SIN_MODELO=1 make bootstrap` .

> **Guarda las huellas SHA-256 que imprime**
> Compártelas con los participantes antes del evento. Es la forma de saber que todos trabajan sobre el mismo archivo — y de paso es una lección de cadena de custodia que sale gratis.

3 Verificar la cadena

2 min

```
./scripts/verify-setup.sh
```

```
[ok] tshark encontrado (TShark (Wireshark) 4.2.2)
[ok] evidencia presente (896 paquetes)
[ok] disector TriStation cargado
[ok] protocolo detectado: TRISTATION
[ok] deteccion TRITON en 2 tramas
[ok] ollama encontrado
[ok] servicio de ollama respondiendo en localhost:11434
[ok] modelo afinado 'triton-analista' disponible

LISTO — el taller es viable en esta maquina.
```

No avances sin todas las líneas `[ok]` . Todo lo que sigue depende de que el disector esté cargado. Si alguna falla, ve a la tarjeta de fallas al final de esta guía.

Comprobación rápida de que la evidencia es la correcta

```
make stats     # 896 paquetes, 435 s, 192.168.1.88 ↔ 192.168.1.2
make triton    # debe devolver las tramas 75 y 887
```

4 El modelo local y el cliente MCP

30 min

> **Todo el análisis corre dentro de la VM**
> Sin llamadas a proveedores externos, sin cuentas y sin tokens. Durante el taller no hace falta internet, y la evidencia nunca sale de la máquina — que en un taller de forense OT no es un detalle menor.

> **Verificado de punta a punta**
>
> La cadena completa — `ollmcp` → `wireshark-mcp` → `tshark` del sistema → `TriStation.lua` — se corrió en una VM Ubuntu 24.04 y funciona. **El disector se hereda**, que era la incógnita que decidía si este taller era viable: el modelo ve `TRISTATION`, no `UDP`. Aun así, valídala tú en tu máquina: el hardware cambia.

4.1 Ollama y el modelo

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen3.5:4b     # 3.4 GB, tarda

# afinar: los valores por omisión de Ollama rompen este laboratorio
cd ~/taller                # el Modelfile es ruta relativa al repo
ollama create triton-analista -f setup/Modelfile
ollama list                # debe aparecer triton-analista
```

> **Por qué hay que afinarlo**
>
> Ollama trae `num_ctx` en **2048 tokens** y `temperature` en **0.8**. Con 2048 la salida de `tshark` desborda el contexto y el modelo pierde la evidencia a media sesión; con 0.8 fabrica justo la confabulación que el taller enseña a detectar. `setup/Modelfile` los lleva a 8192 y 0.1, con seed fijo para que el análisis sea reproducible.

Por qué este modelo y no uno más grande. El laboratorio no le pide al modelo que escriba bien: le pide que **invoque `tshark`, lea el resultado y no invente entre una llamada y otra**. En la evaluación de tool calling de 2026 sobre 40 casos deterministas, Qwen3.5 4B obtuvo **97.5% de acierto** — primer lugar, por encima de modelos cinco veces mayores, en 3.4 GB. Ahí es donde los modelos locales se separan.

4.2 El cliente y el servidor MCP

```
sudo apt install -y pipx     # NO viene instalado por omisión
pipx install ollmcp          # el cliente
pipx install wireshark-mcp   # el servidor que expone tshark
pipx ensurepath
exec $SHELL                  # sin esto: "ollmcp: command not found"

cd ~/taller
mkdir -p ~/.config/ollmcp
cp setup/ollmcp-config.json ~/.config/ollmcp/config.json
ollmcp -j setup/mcp-config.minimo.json
```

> **Tres trampas en una sola línea**
>
> **El flag es `--servers-json` (`-j`)**, no `--mcp-server`: ése existe, pero espera la ruta a un script `.py` o `.js`, y con el nombre de un binario responde `Server script not found`.
> **El servidor es `wireshark-mcp`, no `mcp-wireshark`.** El segundo instala limpio y no arranca: revienta con `AttributeError: 'Server' object has no attribute 'list_tools'`.
> **Y no pases `-m`.** Ver el recuadro siguiente — es la peor de las tres.

La trampa del `-m`: arranca el modelo equivocado en silencio

`ollama` etiqueta el modelo como `triton-analista:latest` y `ollmcp` compara la cadena completa. Con `-m triton-analista` no encuentra coincidencia, avisa en **una sola línea** y arranca con el modelo base:

```
Requested model triton-analista is not available — using qwen3.5:4b instead.
```

El laboratorio corre igual: las herramientas funcionan, el disector se hereda, las respuestas se ven bien. Lo que se perdió es el `SYSTEM` del `Modelfile` — sin reglas de anclaje, sin etiquetas, sin "una herramienta a la vez". Es decir, **exactamente lo que el taller mide**.

El `config.json` ya trae el modelo, así que lo seguro es **omitir el flag**. Y antes de cada sesión, mira el prompt del cliente: debe decir `triton-analista/…`, no `qwen3.5/…`.

Qué trae `ollmcp-config.json`

Los parámetros verificados y, sobre todo, la **curaduría de herramientas: 9 habilitadas de 52**. Las 43 apagadas incluyen toda la detección automática del servidor — `detect_beaconing`, `detect_port_scan`, `extract_credentials`, `yara_scan` — que sobre esta captura no produce hallazgos: produce ruido plausible. El propio servidor recomienda tres de ellas al abrir el archivo, y un modelo pequeño sigue recomendaciones. La curaduría es lo que impide que el taller se vuelva una cacería de falsos positivos.

Con pipx, no con pip

Desde Ubuntu 24.04 rige PEP 668: `pip install --user` falla con `externally-managed-environment` porque el Python del sistema lo gestiona apt. `pipx` es además la vía correcta para una aplicación de línea de comandos — entorno aislado y binario en el PATH.

4.3 Fijar los parámetros dentro del cliente

Este paso no es opcional, y es el que más fácil se salta. `ollmcp` lleva su **propia** configuración de modelo. Aunque hayas construido `triton-analista` con el `Modelfile`, al abrir el panel de parámetros puedes encontrarte esto:

```
/mc

num_ctx        -
temperature    -
seed           -
```

Un guión no quiere decir 8192, ni 0.1: quiere decir que **el cliente no está enviando ese parámetro**. Con `num_ctx` sin fijar vuelves al problema de raíz — el modelo se queda sin contexto a media línea de tiempo y, como no sabe decir "se me olvidó", rellena. Fíjalos ahí mismo y guárdalos:

Parámetro	Valor
`num_ctx`	`8192`
`temperature`	`0.1`
`seed`	`42`
`/sc`	Guarda en `~/.config/ollmcp/config.json`

> **Ese archivo tiene que ir dentro de la OVA**
> Si exportas la imagen sin `~/.config/ollmcp/config.json`, cada participante arranca con los parámetros en `-` y el laboratorio se comporta distinto en cada máquina — que es justo lo que la snapshot existe para evitar. Confírmalo con `cat` antes del paso 5.

4.4 La pregunta de control

El mismo día del taller, antes de que llegue nadie:

```
Corre: tshark -r malware_exec.pcap -c 1
y dime qué protocolo reporta.
```

> **Debe responder TRISTATION — y debes ver la llamada a la herramienta**
> Hay dos formas de fallar y se arreglan distinto. Distínguelas antes de seguir.

Qué ves	Qué pasa	Qué hacer
Responde `UDP`	El disector no cargó: el proceso corre como root o sin el plugin	Tarjeta de fallas, primera fila
Responde algo plausible **sin llamar la herramienta**	El modelo está inventando	**Cambia de modelo. No sigas.**

El segundo caso es el grave: es exactamente el fallo que el taller entero enseña a detectar. Si el modelo lo hace en la validación, lo va a hacer toda la sesión.

Cuando responde bien, la prueba dura de que el disector llegó hasta el modelo es el campo `frame.protocols`:

```
"frame.protocols": "eth:ethertype:ip:udp:tristation"
```

Si termina en `udp`, la cadena está rota aunque el modelo suene convincente.

4.5 Lo que vas a ver, y no significa lo que parece

Tres cosas de la *herramienta* —no del caso— que salen en cuanto empiezas a consultar. Ninguna es un problema; todas confunden si te agarran en frío:

- Las herramientas se listan con `/tools`, y **su numeración cambia entre versiones del servidor**. No memorices números ni los pongas en el material del participante: lo que se dicta en voz alta es el nombre.
- Pedir la lista de paquetes con `limit: 1` devuelve **cabeceras, no contenido**. Para ver los campos que decodifica el disector, la vía sigue siendo `tshark -r <archivo> -c 1 -V`.
- El resumen de *expert info* da **frecuencias, no números de trama**. Sirve para saber *qué* aparece y *cuántas veces*; para saber *dónde*, hace falta otra consulta.

4.6 Cuánto tarda

En la VM de referencia, cada consulta —la ida y vuelta completa: el modelo decide, llama la herramienta, lee el resultado y responde— tardó alrededor de **tres minutos** sobre CPU.

> **Mídelo en tu máquina y dilo en voz alta**
> Ese número depende por entero del equipo: núcleos, RAM y si hay GPU. Sin el aviso, alguien da por muerto el cliente a los veinte segundos y lo reinicia a media consulta, perdiendo todo el contexto que llevaba anclado. El corolario que también conviene decir: **cada consulta cuesta**, así que vale más pensar la pregunta que dispararla.

4.7 Si el modelo local no da el ancho

No lo fuerces. La ruta con un proveedor en la nube sigue siendo válida (`SIN_MODELO=1 make bootstrap`). Pierdes la independencia de la red del venue y ganas capacidad. **Decídelo en tu ensayo, no en el salón.**

5 El guion de la sesión —

Lo que se pega en el cliente, en orden. Cada consulta trae qué debe llamar el modelo, qué debe salir, y **el comando con el que tú lo compruebas** mientras circulas por la sala. Los mismos prompts sin las respuestas están en `prompts/01-baseline.md` — ése es el que se reparte.

> **Antes de la primera consulta va `00-reglas-de-anclaje.md`**
> Sin excepción, y aunque el `Modelfile` ya las traiga en su `SYSTEM`: en el ensayo el modelo respondió sin una sola etiqueta hasta que se le recordaron. Un 4B con razonamiento encendido gasta su obediencia en pensar, no en formatear.

Las nueve herramientas, y su equivalente en tshark

Herramienta	Qué hace	En `tshark`
`wireshark_open_file`	Abre la captura y lista los protocolos. La puerta de entrada	`-z io,phs`
`wireshark_get_file_info`	Tramas, duración, tamaño y **SHA-256**	`capinfos`
`wireshark_stats_protocol_hierarchy`	Jerarquía de protocolos con conteo por capa	`-q -z io,phs`
`wireshark_stats_conversations`	Quién habla con quién, con paquetes y bytes	`-q -z conv,ip`
`wireshark_stats_expert_info`	Lo que el disector marca como error, aviso o nota	`-q -z expert`
`wireshark_get_packet_list`	Lista de tramas — el panel de arriba de Wireshark	`-T fields`
`wireshark_get_packet_details`	Una trama, decodificada entera	`-c 1 -V`
`wireshark_extract_fields`	Extrae campos concretos, en tabla	`-T fields -e campo`
`wireshark_search_packets`	Busca por contenido: texto, hex o regex	`-Y 'frame contains'`

> **Los tres parámetros que cambian todo**
> **`display_filter`** filtra *antes* de devolver — sin él llegan 896 líneas, casi todas vacías. **`custom_columns`** elige las columnas, y convierte la línea de tiempo en **una sola llamada**. **`limit`** por omisión es 20 en `get_packet_list` y 100 en `extract_fields`: es lo que produce el `[Showing 100/897 lines]` del que el modelo generaliza. **Enseña a pedir los tres explícitos.**

Bloque 1 — Reconocimiento

```
Abre /home/analista/taller/evidence/tricotools/malware_exec.pcap.
No te voy a decir de qué entorno viene ni qué protocolo esperar.

Dime cuántos paquetes trae, cuánto dura la captura y su SHA-256.
Nombra la herramienta que usaste.
```

Debe salir: 896 tramas · 435.39 s · `3fd16a17…ad90` · `tristation` entre los protocolos. **Compruébalo:** `capinfos $PCAP`

```
¿Qué hosts participan y en qué puertos? ¿Cuál dirías que es el cliente
y cuál el dispositivo? Justifícalo con algo medible y marca la
justificación como [INFERENCIA].
```

Debe salir: `192.168.1.88` ↔ `192.168.1.2`, UDP **1502**. **Compruébalo:** `tshark -r $PCAP -q -z conv,ip`

Bloque 2 — La forma del tráfico

```
Dame la jerarquía de protocolos de la captura.
```

Debe salir: `tristation` en 896, `ts.crc16` en **895**, `text` en **1**. **Compruébalo:** `tshark -r $PCAP -q -z io,phs`

> **Dos asimetrías que ningún equipo mira**
> Una trama de 896 no tiene CRC y trae una capa de texto — casi con seguridad la misma que dispara el error de Lua del disector. Siembra la pregunta: *¿por qué 895 y no 896?*

```
Extrae el campo ts.ts_cmd usando display_filter "ts.ts_cmd" y limit 900.
Dame la distribución de valores ordenada por frecuencia.
Si la salida viene truncada, dímelo en lugar de generalizar.
```

Debe salir: `19` y `108` dominan — **676 tramas, el 75%**. **Compruébalo:** `tshark -r $PCAP -T fields -e ts.ts_cmd | sort -n | uniq -c | sort -rn`

```
Dos comandos dominan y aparecen emparejados. Dime cuáles son, qué
porcentaje representan y qué hipótesis tienes sobre su función.
Marca la hipótesis como [INFERENCIA].

Después dime cuántas tramas quedan si los filtras, y qué comandos
aparecen en ellas.
```

Bloque 3 — La secuencia

```
Dame la lista de paquetes con:
  display_filter: ts.ts_cmd && ts.ts_cmd!=19 && ts.ts_cmd!=108
  custom_columns: frame.number,frame.time_relative,ip.src,ts.ts_cmd
  limit: 250

Ésa es la línea de tiempo sin el ruido de fondo.
```

> **Ésta es la consulta que enseña la técnica**
> Filtrar y elegir columnas convierte lo que serían cinco consultas de tres minutos en una. Si un equipo aprende sólo una cosa técnica del taller, que sea ésta.

```
Agrupa esas tramas en fases por comportamiento y ponle nombre a cada una.
Cada fase con trama de inicio, trama de fin y tiempo relativo.
No inventes fases que no se sostengan en las tramas que ya viste.
```

```
¿Hay un patrón de comandos que se repita y que parezca marcar el inicio
de una operación de escritura? Muéstrame todas sus ocurrencias con
números de trama, y para cada una, qué respondió el dispositivo.
```

```
Dame las últimas 10 tramas de esa línea de tiempo filtrada.
¿Cuál es el último comando significativo que envía el cliente, y qué
respondió el dispositivo exactamente? Léelo antes de interpretarlo.
```

Debe salir: el *Run program* final **fue rechazado**. **Tu chuleta:** `tshark -r $PCAP -Y 'frame.number>=889 && frame.number<=892' -T fields -e frame.number -e ip.src -e ts.ts_cmd` — no dictes esos números a la sala.

Bloque 4 — El estado del controlador

```
El comando de respuesta más frecuente trae información de estado.
Toma una trama de ésas y dame TODOS los campos que el disector decodifica
en ella, con su valor.
```

Compruébalo: `tshark -r $PCAP -Y 'ts.ts_cmd==108' -c 1 -V` — es el comando que revela qué campos existen, y sin él nadie sabe qué pedir.

```
¿Alguno de esos campos describe el estado de un control físico del equipo?

Extrae ts.ts_cp_keyState y ts.ts_cp_runState con:
  display_filter: ts.ts_cp_keyState
  limit: 400

Dime en cuántas tramas aparece y si el valor cambia a lo largo de la
captura. Si la salida viene truncada, dilo en lugar de generalizar.
```

Debe salir: 338 tramas, `keyState = 0x01` constante. **Compruébalo:** `tshark -r $PCAP -Y 'ts.ts_cp_keyState' -T fields -e ts.ts_cp_keyState -e ts.ts_cp_runState | sort | uniq -c`

```
Ese valor: ¿qué implica operativamente? Marca claramente qué parte de tu
respuesta es [EVIDENCIA] de esta captura y qué parte es [CONOCIMIENTO] de
tu entrenamiento sobre este tipo de equipo.
```

> **Aquí se cierra el taller — y aquí el modelo falla**
>
> En el ensayo tradujo *key* como **tecla** —"estado de la tecla, 0x01 = presionada"— en vez de **llave física**. Y lo etiquetó `[CONOCIMIENTO]`, correctamente. El disector entrega el campo; el significado lo pone el humano. Deja que la sala lo vea antes de abrir la rúbrica.

Los prompts que NO se deben usar

Prompt	Por qué arruina el ejercicio
"¿Qué malware es este?"	Invita al modelo a recitar entrenamiento
"Resume qué pasó"	Produce narrativa sin anclaje
"¿Es esto TRITON?"	Pregunta cerrada: dirá que sí y construirá la evidencia hacia atrás
"Dame los IOCs"	No hay IOCs útiles aquí; pedirlos es invitar a que los invente
"Corre un análisis de seguridad completo"	Dispara la detección automática, que sobre tráfico industrial devuelve ruido plausible

La pregunta cerrada obtiene la respuesta que sugiere. En forense, la diferencia entre un buen prompt y uno malo es la misma que entre una buena y una mala pregunta en un interrogatorio.

Las tres preguntas de control

Para el participante, cuando algo le suene demasiado redondo. Cuestan una consulta y evitan meter una invención en el reporte.

```
¿De qué comando exacto salió esa afirmación? Pégame la salida cruda.

Ese número de trama, ¿lo leíste de una columna de números de trama, o de
otra columna? Enséñame la línea de donde lo sacaste.

¿Eso es una propiedad del tráfico capturado, o del programa que lo está
decodificando?
```

La tercera es la que más veces va a cambiar un reporte. Una pista rápida para contestarla: **si algo aparece en el 100% de las tramas, es del instrumento, no del incidente.**

6 Snapshot

5 min

Con la verificación en verde y el MCP respondiendo TRISTATION, limpia y congela:

```
cat ~/.config/ollmcp/config.json    # PRIMERO: confirma que existe
history -c
sudo apt clean
sudo rm -rf /tmp/*
```

Y toma un snapshot del hipervisor llamado `baseline-verificada`.

Es tu red de seguridad: si un participante destruye su entorno a media sesión, reviertes en treinta segundos en lugar de perderlo. **Tómala después de verificar, nunca antes.**

7 Ensayar

2 h

El paso que todo mundo se salta y del que todo mundo se arrepiente. Corre el workbook completo tú mismo, con cronómetro, como si fueras participante — sin abrir la rúbrica.

- [ ] ¿Alcanzan los 30 minutos del Bloque 3?
- [ ] ¿Llegaste solo a la trama 892, o necesitaste la pista?
- [ ] ¿Encontraste el `keyState`, o te lo dijo el modelo de memoria?
- [ ] ¿Cuántas afirmaciones de tu asistente no pudiste anclar a una trama?
- [ ] ¿Cuánto tardó cada consulta **en esta máquina**? Ése es el número que dices en voz alta.

Después contrasta contra `docs/03-rubrica.md`. Lo que se te dificultó a ti se le va a dificultar a la sala, y ahí es donde ajustas las pistas.

Los tres modos de falla — verificados, y reproducibles

Todo lo que sigue pasó de verdad en el ensayo, con el modelo afinado, el disector cargado y `seed 42`. Si corres las mismas consultas, sale lo mismo. La tesis, en una frase:

> **El modelo extrae bien y explica mal.**
>
> Lo que saca del comando suele ser correcto; lo que dice que *significa* es donde inventa. Y las dos cosas llegan en el mismo párrafo, con el mismo tono de seguridad.

	Qué pasa	Cómo se detecta
A	Un artefacto de la **herramienta** leído como propiedad del **tráfico**	¿Esto lo produjo la red, o el decodificador?
B	Una **columna mal leída** convertida en dato duro	Vuelve a la salida cruda y busca el número
C	Una **etiqueta mal puesta** — `[EVIDENCIA]` sobre lo inventado	Pide el comando que lo respalda

Familia A — el artefacto de la herramienta

Al pedir el resumen de *expert info*, la herramienta devuelve:

```
Errors (3)
   Frequency      Group          Protocol  Summary
           1  Undecoded        TRISTATION  Lua Error: .../TriStation.lua:611: negative length in tvb
range
           2  Malformed        TRISTATION  TRITON malware detected!

Notes (2)
   Frequency      Group          Protocol  Summary
           2  Protocol         Ethertype  Didn't find padding of zeros, and an undecoded trailer
exists.
```

Y al pedir la jerarquía de protocolos:

```
udp                    frames:896
  _ws.lua.fake         frames:896
    tristation         frames:896
      ts.crc16         frames:895
      text             frames:1
```

Lo que leyó	Lo que concluyó	Qué es en realidad
`TRITON malware detected!`	Tramas con firma de TRITON	**Correcto.** Es el hallazgo, y viene del disector
`Lua Error: TriStation.lua:611`	"Corrupción o manipulación del protocolo"	Un **bug del disector**, de 2018. Dispara una vez, siempre la misma trama. `Undecoded` es "no pude leerlo", no "está dañado"
`Didn't find padding of zeros`	Posible "modificación maliciosa del paquete"	Aviso normal de Wireshark sobre el relleno de la trama Ethernet
`_ws.lua.fake`	Posible **"inyección de código"**	La capa puente que Wireshark inserta para un disector escrito en Lua. La creó `TriStation.lua`

> **El argumento que mata a los tres de un golpe**
>
> `_ws.lua.fake` aparece en **896 de 896 tramas**. Si fuera inyección, estaría también en las 676 tramas de *polling* benigno. Un indicador presente en el 100% de una captura no distingue nada — y un hallazgo que no separa señal de ruido no es un hallazgo: es el instrumento.

Y dos números que el modelo no comentó: `ts.crc16` en 895 y `text` en 1. Una trama de 896 no tiene CRC y trae una capa de texto — casi con seguridad la misma que dispara el error de Lua. Excelente pista para el Bloque 2, visible a simple vista, y ningún equipo la mira.

Familia B — la columna mal leída

De esa misma salida, el modelo escribió:

```
[EVIDENCIA] Error 2 (Malformed, TRISTATION): "TRITON malware detected!" (frame 2)
```

> **Las tramas son la 75 y la 887**
> La columna que leyó dice `Frequency`: es *cuántas veces* ocurre, no *dónde*. Se comprueba en un comando — `make triton`. Un número de trama **inventado**, con etiqueta de evidencia, dentro de un reporte por lo demás correcto. **Si sólo puedes mostrar un ejemplo en toda la sesión, muestra éste.**

Regla operativa: el resumen de *expert info* da **frecuencias, no números de trama**. Para saber *dónde*, hace falta otra consulta.

Familia C — la etiqueta mal puesta

En la misma respuesta, el modelo etiquetó:

- `[CONOCIMIENTO] El disector ha detectado anomalías` — eso salió de un comando. Es `[EVIDENCIA]`.
- `[EVIDENCIA] ... (frame 2)` — eso no salió de ningún lado. Es invención.

> **Etiquetó al revés — y ésa es la lección que cierra el taller**
> Las etiquetas no *son* la verificación: son el sitio donde se aplica. Un reporte con `[EVIDENCIA]` en todas partes no está anclado, está maquillado. Lo que ancla es que puedas correr el comando y ver la trama.

El caso del `keyState` — extrae bien, interpreta mal

Es el más importante, porque ocurre justo sobre el hallazgo del taller. Pedido de extraer `ts.ts_cp_keyState`, el modelo acertó lo medible — **`0x01` constante**— y después escribió:

```
[CONOCIMIENTO] ts.ts_cp_keyState: Estado de la tecla (0x01 = presionada/activa)
```

No es una tecla. Es la **llave física** del selector del Triconex, y `0x01` es la posición **PROGRAM**. Traducir *key* como *tecla* en vez de *llave* destruye el hallazgo entero: convierte el control que habría detenido el incidente en un detalle de teclado.

Dos cosas que enseñar con esto:

1. **La etiqueta funcionó.** El modelo marcó esa frase como `[CONOCIMIENTO]` — "esto no lo verifiqué aquí". El sistema avisó bien, y aun así el analista se lo traga si no lee las etiquetas.
2. **El modelo no sabe TriStation, y no puede saberlo.** El disector le entrega el campo; el significado lo pone el humano. Es la tesis del taller en una línea, demostrada sobre el hallazgo central.

Además generalizó desde datos truncados: la herramienta devolvió `[Showing 500/897 lines]` y el modelo afirmó "en todas las tramas" como `[EVIDENCIA]`. Nunca contó las 338. Es la familia B en su forma más común: **afirmar más de lo que el comando devolvió**.

> **Nota práctica**
> `extract_fields` devuelve una línea por trama, vacías incluidas: 896 líneas de las que sólo 338 traen dato. Se desperdicia contexto y se invita a la generalización. Enseña a filtrar antes de extraer.

El A/B que vale toda la sesión

Si tienes cinco minutos al cierre, corre la **misma** pregunta contra el modelo base y contra `triton-analista`. Sin el `SYSTEM`, la respuesta a "¿qué protocolo reporta la primera trama?" fue:

```
El protocolo principal que reporta la primera trama es UDP
```

…teniendo `"frame.protocols": "eth:ethertype:ip:udp:tristation"` en la mano. Enterró el hallazgo bajo el modelo OSI, añadió "el puerto 1502 es típicamente utilizado por el servicio Tris" —memoria de entrenamiento presentada como hecho, sin etiquetar— y encadenó tres herramientas sin leer entre una y otra.

Misma evidencia, mismas 9 herramientas, mismo hardware. Lo único distinto son las reglas. Proyectadas lado a lado, nadie vuelve a preguntar para qué sirve el anclaje.

Graba el video de respaldo

Con el modelo en local ya no dependes de la red del venue, así que la falla más común desapareció. Pero el video sigue sin ser opcional, por otra razón: **un modelo pequeño puede portarse mal en vivo**. Si eso pasa, el video te deja seguir dando el taller como demo guiada en lugar de perderlo.

8 Semana del taller —

- [ ] Publicar el brief y la guía de la VM **48 horas antes** — que nadie descargue nada en el salón
- [ ] Comunicar las huellas SHA-256 a los participantes
- [ ] Exportar la OVA y copiarla a **dos memorias USB** (no la distribuyas por el WiFi del venue)
- [ ] Imprimir el workbook — hay quien trabaja mejor en papel
- [ ] Confirmar que las laptops de los participantes pueden asignar **8 GB** a la VM
- [ ] Validar la pregunta de control **el mismo día** — el modelo debe responder TRISTATION
- [ ] Revisar `/mc` el mismo día: ningún parámetro en `-`
- [ ] **Confirmar que el prompt dice `triton-analista`**, no `qwen3.5`
- [ ] Confirmar `9/52 tools enabled` en el arranque del cliente
- [ ] Tener a la mano el tiempo por consulta medido, para anunciarlo al abrir
- [ ] Preparar **dos mesas de rescate** con el entorno ya corriendo
- [ ] Probar la VM en una máquina **que no sea la tuya**

9 Tarjeta de fallas

Síntoma	Causa	Solución
Protocolo aparece como `UDP` y payload `data`	Corriendo como root: Wireshark deshabilita Lua para superusuario	Usar el usuario sin privilegios. Nunca `sudo`.
`tshark -G protocols | grep tristation` no devuelve nada	El disector no cargó	Confirmar la ruta con `tshark -G folders`, o usar `-X lua_script:`
Las huellas SHA-256 no coinciden	Alguien trabaja sobre otro archivo	Volver a correr `make evidence` y comparar antes de seguir
El asistente responde `UDP` a la pregunta de validación	El proceso del MCP corre como root o con otro usuario	Revisar bajo qué usuario arranca el MCP
El asistente responde sin llamar la herramienta	El modelo está inventando en lugar de consultar	Cambiar de modelo. No es recuperable con prompts.
`pipx: command not found`	No viene instalado en Ubuntu por omisión	`sudo apt install -y pipx`
`ollmcp: command not found` tras instalarlo	Quedó en `~/.local/bin`, fuera del PATH de la sesión	`pipx ensurepath && exec $SHELL`
`pip`: `externally-managed-environment`	PEP 668 en Ubuntu 24.04	Usar `pipx` para aplicaciones, `apt` para librerías
`no Modelfile or safetensors files found`	Ruta relativa: no estás dentro del repositorio	`cd ~/taller` antes de `ollama create`
`ollama` no responde o el modelo no carga	Servicio caído, o RAM insuficiente en la VM	`ollama ps`. Subir la RAM a 8 GB o más.
Cada consulta tarda ~3 minutos	**Es normal en CPU.** No es una falla	Anúncialo al abrir la sesión. Más vCPU y RAM lo bajan; una GPU, mucho más.
El prompt dice `qwen3.5/…`	Pasaste `-m` con la etiqueta incompleta: corre el modelo **base**, sin reglas de anclaje	Salir y entrar **sin** `-m`. Es la falla más silenciosa del lab
El modelo dice "(frame 2)" sobre *expert info*	Leyó la columna `Frequency` como número de trama	No se arregla: **es material del taller.** Son la 75 y la 887
El modelo habla de "estado de la tecla"	Tradujo *key* como tecla, no como llave física	No se arregla: **es la tesis del taller.** Ver la sección 7
`Server script not found`	Usaste `--mcp-server`: espera un `.py` o `.js`	`-j setup/mcp-config.minimo.json`
`AttributeError: 'Server' object has no attribute 'list_tools'`	Es `mcp-wireshark`: incompatible con el SDK actual de MCP	`pipx install wireshark-mcp` y cambiar el `command` del JSON
En `/mc` los parámetros aparecen como `-`	El cliente no los está enviando: el `Modelfile` no basta	Fijarlos en `/mc` y guardar con `/sc`
El modelo reporta "paquetes corruptos"	Leyó el bug de Lua del disector como propiedad del tráfico	No se arregla: **es material del taller.** Ver la sección 7

El asistente inventa IOCs o dominios	Falta la regla de anclaje, o el disector no está cargado	Pegar `prompts/00-reglas-de-anclaje.md` antes de cualquier pregunta
El conteo de paquetes no es 896	PCAP incorrecto o truncado	`make clean && make evidence`

10 Referencia rápida

Comando	Qué hace
`make bootstrap`	Provisiona la VM de cero a verificada
`make evidence`	Descarga la evidencia desde el origen
`make verify`	Valida la cadena completa
`make stats`	Resumen de la captura
`make triton`	Tramas donde el disector detecta TRITON
`make clean`	Borra la evidencia descargada
`make hooks`	Activa el hook de pre-commit
`make check-docs`	Verifica que no falte documentación
`ollama list`	Modelos descargados
`ollama ps`	Si el modelo está cargado en memoria
`ollmcp -j setup/mcp-config.minimo.json`	Arranca el cliente. **Sin `-m`**

Dentro del cliente

Comando	Qué hace
El prompt del cliente	Debe decir `triton-analista/…`. Si dice `qwen3.5/…`, sal y entra sin `-m`
`/mc`	Panel de parámetros del modelo. **Ninguno debe decir `-`**
`/sc`	Guarda la configuración en `~/.config/ollmcp/config.json`
`/tools`	Herramientas disponibles. La numeración cambia entre versiones

Los números que debes reconocer de memoria

Dato	Valor
Paquetes / duración	896 / 435.39 s
Hosts	192.168.1.88 (estación) ↔ 192.168.1.2 (Triconex), UDP 1502
Ruido de fondo	676 tramas (75%) de *Get CP status*
Detecciones de TRITON	Tramas 75 y 887
El detalle que casi nadie ve	Trama 891 *Run program* → trama 892 **Command rejected**
El hallazgo	`keyState = 0x01 [Program]` constante en las 338 respuestas
Lo que **no** es del caso	El `Lua Error` de `TriStation.lua:611`, el aviso de *padding* de Ethernet y `_ws.lua.fake`: artefactos de la herramienta
Las dos asimetrías de la jerarquía	`ts.crc16` en 895 y `text` en 1 — una trama sin CRC, con capa de texto
La prueba de que el disector llegó al modelo	`frame.protocols` termina en `:tristation`

Guía de secuencia del laboratorio · Repositorio `cyberforcesummit2026_workshop` · Documentación completa en `docs/` · Evidencia y disector: Nozomi Networks, BSD-3-Clause